KITCHENHUB

Home Made Food

Software Project Documentation & System Specification

Project Status: LIVE / PRODUCTION

Business: Home Made Food

Location: Bahawalpur, Pakistan

Repository: arslan-arif6/Food_0dering-platform

Live Site: https://homemadefood-pk.vercel.app/

Prepared: August 2026

Author: Arslan Arif

Table of Contents

1. Executive Summary

2. Objectives & Scope

3. Users, Roles & Business Rules

4. Functional Features

5. System Architecture

6. Technology Stack

7. Data & Database Model

8. Security Model

9. Storage & Media Security

10. Order & Notification Workflow

11. UX, Performance & SEO

12. Testing & Quality Assurance

13. Deployment & Production Operations

14. Known Limitations

15. Backup & Recovery

16. Future Scope

17. Maintenance Guidance

18. Conclusion

1. Executive Summary

KitchenHub is a mobile-first online ordering platform built for Home Made Food, a local home-kitchen business in Bahawalpur. It replaces manual WhatsApp-based order handling with a structured customer ordering experience and a dedicated admin dashboard.

Customers browse the menu, select variants, manage a cart, complete checkout, place orders and track status. Authorized staff manage orders, menu data, categories, customers, restaurant settings and operations from the admin dashboard.

The system is intentionally scoped as a single-restaurant solution, prioritizing maintainability, security, mobile usability and production readiness over unnecessary enterprise complexity.

2. Objectives & Scope

2.1 Objectives

- Digitize manual order taking.
- Provide a clear mobile-first menu and checkout.
- Give the owner a centralized order-management dashboard.
- Protect administrative operations with authentication, MFA and authorization.
- Enforce security at both application and database layers.
- Operate as a real deployed production service.

2.2 Scope

- Single restaurant / home kitchen.
- Single owner with owner/admin role separation.
- Approximately 30 menu items.
- Mobile-first customer experience.
- Cash on Delivery, JazzCash and Easypaisa payment options.

2.3 Explicitly Out of Scope

- Multi-restaurant / marketplace support
- Multi-branch management
- Rider/driver assignment
- Microservices infrastructure
- Full coupon engine

3. Users, Roles & Business Rules

Item	Specification
Customer	Browse, cart, checkout, order and tracking.
Admin	Authorized operational management of supported business data and orders.
Owner	Highest administrative role; owner account requires MFA.
Restaurant model	Single Home Made Food business.
Payments	COD, JazzCash and Easypaisa; automated

	gateway verification is not represented as implemented.

4. Functional Features

4.1 Customer Features

Feature	Description
Home	Business information, featured dishes, categories, hours and contact paths.
Menu	Browse available homemade dishes.
Categories	Breakfast, lunch and dinner organization.
Dish variants	Half/full portions where configured, with dynamic pricing.
Cart	Add, remove and adjust items.
Checkout	Delivery area, address, notes and order information.
Delivery logic	Delivery fees, minimum-order rules and free-delivery threshold logic.
Availability	Blocks unavailable dishes and ordering outside configured business constraints.
Order confirmation	Confirms successful order submission.
Order tracking	Shows order status after submission.
Payments	COD, JazzCash and Easypaisa options.

4.2 Admin Features

Feature	Description
Login	Protected administrative authentication.
MFA	Mandatory for owner; admin enrollment behavior is supported.
Role separation	Owner/admin authorization model.
Order management	Receive and update order status.
Menu/categories	Create, update and delete supported menu data.
Customers	Administrative customer records.
Restaurant settings	Hours, delivery zones/fees, minimum order, payment methods and restaurant information.
Analytics	Basic order-related business analytics.
Dish media	Admin-controlled image upload/update/delete.
New-order notification	Realtime notification and sound for an active admin dashboard.
WhatsApp action	Operational WhatsApp action available from the order workflow.

5. System Architecture

```
Customer
  ↓
Next.js 16 Application
  ├─ App Router / React / TypeScript
```

```
  ├─ Server & Client Components
  ├─ Server Actions
  └─ Validation + Auth/Authorization
  ↓
Supabase
  ├─ PostgreSQL
  ├─ Auth + MFA
  ├─ Row Level Security
  ├─ Storage
  └─ Realtime
  ↓
Vercel Production
```

5.1 Administrative Security Path

```
Request → Authentication → Admin/Owner Authorization → Server Action → Supabase RLS → PostgreSQL
```

Defense in depth is deliberate: server-side authorization is combined with database-level RLS instead of relying on frontend visibility alone.

6. Technology Stack

Layer	Technology
Application	Next.js 16 App Router
UI	React 19, TypeScript, Tailwind CSS
Forms/validation	React Hook Form, Zod
UI utilities	Lucide React, Sonner
Backend/data	Supabase
Database	PostgreSQL
Auth	Supabase Auth + MFA
Authorization	RLS + server-side authorization
Storage	Supabase Storage
Realtime	Supabase Realtime
Deployment	Vercel

7. Data & Database Model

Entity	Purpose
admins	Administrative identity, role and active status.
customers	Customer records for ordering operations.
categories	Menu category organization.
dishes	Menu item data, availability, prices and variants.
orders	Order header, status, payment and delivery information.
order_items	Line items belonging to an order.
restaurant_settings	Business hours, delivery, payment and restaurant configuration.

Orders and order_items form the core transaction relationship: one order can contain multiple line items.

8. Security Model

Security is treated as a core architectural requirement. Authentication, role authorization, database RLS, storage policies and server-side checks work together.

- Supabase Auth provides authenticated identity.
- Owner MFA is mandatory.
- Owner and admin roles are separated.
- Admin access checks identity, active status and role.
- Server Actions use an authorization guard before admin mutations.
- RLS independently enforces database access.
- Order pricing is validated/recalculated server-side to prevent client-side price manipulation.

8.1 Authorization Helpers

Control	Purpose
is_admin()	Checks whether the authenticated user is an active administrator.
is_owner()	Checks whether the authenticated user is an active owner.
SECURITY DEFINER	Supports controlled execution of authorization lookup functions.
RLS	Restricts database operations according to authenticated context.

8.2 Secrets

Environment files and secrets must not be committed to the public repository. Supabase service-role credentials must remain server-side and must never be exposed to browser code.

9. Storage & Media Security

Bucket	Access model
dish-images	Public read; admin-only upload/update/delete.
restaurant-assets	Public read; admin-only write/update/delete.

10. Order & Notification Workflow

```
Customer → Menu → Cart → Checkout → Server validation
       → orders/order_items → PostgreSQL
       → Realtime event → Admin dashboard
       → notification + sound → Order processing → Status tracking
```

The implemented notification system is intended for an active admin dashboard. It is not a full Web Push/service-worker push infrastructure; therefore browser/OS suspension or complete browser closure can prevent notification delivery.

11. UX, Performance & SEO

- Mobile-first design.
- Navigation and action loading indicators reduce uncertainty during slower network transitions.
- Form submission and media operations provide loading feedback.
- Realtime order awareness avoids constant polling.
- Production build was verified.
- Metadata includes title, description, Open Graph and Twitter card configuration.
- robots.txt and sitemap.xml are present.
- Favicon and a proper 404 route are configured.

12. Testing & Quality Assurance

Area	Test coverage
Customer	Menu, categories, navigation, cart, checkout, validation, delivery rules, placement, confirmation and tracking.
Admin	Login/logout, protected routes, MFA, orders, status changes, menu/categories, customers and settings.
Security	RLS, storage policies, authorization, unauthorized access and incognito/logout behavior.
Realtime	New-order notification and sound alert.
Production	Vercel deployment, environment behavior, routing, loading and error states.
Mobile	Mobile navigation and real-device interaction.
Real users	Site tested by multiple people before completion.

13. Deployment & Production Operations

```
GitHub → Vercel → Production Web App → Supabase Production Project
```

Item	Value
Live URL	https://homemadefood-pk.vercel.app/
Repository	https://github.com/arslan-arif6/Food_0dering-platform
Deployment	Vercel
Backend	Supabase
Production build	npm run build verified successfully.

Repository changes must be pushed so the connected Vercel deployment can build and publish production updates.

14. Known Limitations

- Single-restaurant architecture.
- No multi-branch or rider/driver management.

- Current notifications are not guaranteed when the browser is completely closed or suspended.
- JazzCash/Easypaisa are payment options; automated gateway verification is not represented as implemented.
- Analytics are intentionally basic.
- Free-plan backup capabilities require a manual backup strategy.

15. Backup & Recovery

- Take regular manual database backups appropriate to order volume.
- Take an additional backup before major schema/security changes.
- Keep a backup outside the live Supabase project.
- Document and periodically validate the restore procedure.
- GitHub source history is not a substitute for database-data backups.

16. Future Scope

Enhancement	Purpose
Web Push	Background/locked-device new-order alerts through push infrastructure.
Online payment gateway	Automated payment processing/verification.
Customer accounts	Persistent customer identity and order history.
PWA	More app-like mobile experience.
Customer reviews	Feedback and social proof.
Advanced analytics	Deeper reporting and trends.
Automated backups	Stronger disaster recovery.
Automated WhatsApp notifications	Reduce manual operational messaging.

17. Maintenance Guidance

- Keep production stable until real usage identifies a need.
- Monitor order flow, notification reliability and admin usability.
- Back up before high-risk database changes.
- Review dependencies and security settings periodically.
- Regression-test RLS/authorization changes.
- Prioritize real business feedback over speculative feature additions.
- Keep secrets out of GitHub.

18. Conclusion

KitchenHub is a completed, deployed small-business ordering platform covering customer ordering, backend persistence, administrative operations, authentication, authorization, database security, media storage, realtime order awareness, testing and production deployment.

Its architecture is deliberately proportional to Home Made Food's needs. The current status is Production / Maintenance; future development should be driven by real operational requirements rather than feature accumulation.

End of Document